

Working with Seed Data

- Due No Due Date
- Points 1
- Submitting a website url

FORK

[_ \(https://github.com/learn-co-curriculum/python-p3-working-with-seed-data/fork\)](https://github.com/learn-co-curriculum/python-p3-working-with-seed-data/fork)  [_ \(https://github.com/learn-co-curriculum/python-p3-working-with-seed-data\)](https://github.com/learn-co-curriculum/python-p3-working-with-seed-data)  [_ \(https://github.com/learn-co-curriculum/python-p3-working-with-seed-data/issues/new\)](https://github.com/learn-co-curriculum/python-p3-working-with-seed-data/issues/new)

Learning Goals

- Use an external library to simplify tasks from earlier ORM lessons.
- Manage database tables and schemas without ever writing SQL through Alembic.
- Use SQLAlchemy to create, read, update and delete records in a SQL database.

Key Vocab

- **Schema:** the blueprint of a database. Describes how data relates to other data in tables, columns, and relationships between them.
- **Persist:** save a schema in a database.
- **Engine:** a Python object that translates SQL to Python and vice-versa.
- **Session:** a Python object that uses an engine to allow us to programmatically interact with a database.
- **Transaction:** a strategy for executing database statements such that the group succeeds or fails as a unit.
- **Migration:** the process of moving data from one or more databases to one or more target databases.
- **Seed:** to fill a database with an initial set of data.

Introduction

This lesson is set up as a code-along, so make sure to fork and clone the lesson. Then run these commands to set up the dependencies and set up the database:

```
# => Installing dependencies from Pipfile.lock (xxxxxx)...\n# => 🐍 ████████████████████████████████████████ X/X - XX:XX:XX\n# => To activate this projects virtualenv, run pipenv shell.\n# => Alternatively, run a command inside the virtualenv with pipenv run.\n# => Launching subshell in virtual environment...\n# => . /python-p3-working-with-seed-data/.venv/bin/activate\n# => $ . /python-p3-working-with-seed-data/.venv/bin/activate
```

In this application, we have two migrations: one for our declarative `Base` , and a second for a table called `games` .

```
class Game(Base):
    __tablename__ = 'games'

    id = Column(Integer(), primary_key=True)
    title = Column(String())
    genre = Column(String())
    platform = Column(String())
    price = Column(Integer())
    created_at = Column(DateTime(), server_default=func.now())
```

```
updated_at = Column(DateTime(), onupdate=func.now())
```

Why Do We Need Seed Data?

With SQLAlchemy, we've seen how simple it is to add data to a database by using built-in methods that will write SQL code for us. For instance, to create a new record in the `games` table, you can open up the Python shell, generate a SQLAlchemy session, create an instance of the `Game` model, and commit it to the session. To simplify this even further, we've used `app/debug.py` to create a session and `import` relevant classes. Run `debug.py` from the `app` directory and enter the following into the `ipdb` shell:

```
botw = Game(title="Breath of the Wild", platform="Switch", genre="Adventure", price=60)
session.add(botw)
session.commit()
```

Awesome! Our database now has some data in it. We can create a few more games:

```
ffvii = Game(title="Final Fantasy VII", platform="Playstation", genre="RPG", price=30)
mk8 = Game(title="Mario Kart 8", platform="Switch", genre="Racing", price=50)
session.bulk_save_objects([ffvii, mk8])
session.commit()
```

► *`bulk_save_objects` is a useful method, but it doesn't carry out all the same tasks as `add`. What attributes will `botw` have that `ffvii` and `mk8` are still missing?*

Since these records are saved in the database rather than in Python's memory, we know that even after we exit the `ipdb` shell, we'll still be able to retrieve this data.

But how can we share this data with other developers who are working on the same application? How could we recover this data if our development database was deleted? We could include the database in version control, but this is generally considered bad practice: since our database might get

quite large over time, it's not practical to include it in version control (you'll even notice that in our SQLAlchemy projects' `.gitignore` file, we include a line that instructs Git not to track any `.sqlite3` files). There's got to be a better way!

The common approach to this problem is that instead of sharing the actual database with other developers, we share the **instructions for creating data in the database** with other developers. By convention, the way we do this is by creating a Python file, `app/seed.py`, which is used to populate our database.

We've already seen a similar scenario by which we can share instructions for setting up a database with other developers: using SQLAlchemy migrations to define how the database tables should look. Now, we'll have two kinds of database instructions we can use:

- Migrations: define how our tables should be set up.
- Seeds: add data to those tables.

Using the `seed.py` File

To use the `seed.py` file to add data to the database, all we need to do is write code that uses SQLAlchemy methods to create new records. Add this to the `seed.py` file below the creation of the `session` object:

```
# app/seed.py

...

botw = Game(title="Breath of the Wild", platform="Switch", genre="Adventure", price=60)
ffvii = Game(title="Final Fantasy VII", platform="Playstation", genre="RPG", price=30)
mk8 = Game(title="Mario Kart 8", platform="Switch", genre="Racing", price=50)

session.bulk_save_objects([botw, ffvii, mk8])
session.commit()
```

To run this code, simply run `python seed.py`. You should not see any output if it executes without error.

Run the `debug.py` script again to enter an `ipdb` shell.

```
session.query(Game).count()
# => 3
session.query(Game)[-1]
# => Game(id=3, title="Mario Kart 8", platform="Switch")
```

Awesome! Exit out of the console.

What happens if we want to add some more data to the database? Well, we could try adding another `add` call in our `seed.py` file:

```
# seed.py

botw = Game(title="Breath of the Wild", platform="Switch", genre="Adventure", price=60)
ffvii = Game(title="Final Fantasy VII", platform="Playstation", genre="RPG", price=30)
mk8 = Game(title="Mario Kart 8", platform="Switch", genre="Racing", price=50)
ccs = Game(title="Candy Crush Saga", platform="Mobile", genre="Puzzle", price=0)
```

Run the seed file again, and let's see our updated data:

```
session.query(Game)[-1]
# => Game(id=7, title="Candy Crush Saga", platform="Mobile")
session.query(Game).count()
# => 7
```

Hmm, we only added four games in the `seed.py` file: why are there now seven games in the database? Well, remember — every time we run `seed.py`, we are creating **new** records in the `games` table. There's nothing stopping our code from producing duplicate data in the database. We're just instructing SQLAlchemy to create new code using this file!

We can modify our seed file to clear our database before each new seed to avoid this complication in the future. Add these commands right after the instantiation of your `session` (but before the creation of any new records):

```
session.query(Game).delete()
session.commit()
```

These commands remove the data from the `games` table and then re-run the seed file. It's handy if you want to start fresh! Just be cautious factoring this into a seed file, because it will inevitably remove all of your data.

We can now see our fresh database with just four records in the `games` table, as intended. Run `python debug.py` :

```
session.query(Game).count()  
# => 4
```

Generating Randomized Data

One challenge of seeding a database is thinking up lots of sample data. Ultimately, when you're developing an application, it's helpful to have realistic data, but the actual content is not so important.

One tool that can be used to help generate a lot of realistic randomized data is the [Faker library](https://faker.readthedocs.io/en/master/index.html%5D) .

<https://faker.readthedocs.io/en/master/index.html%5D>. This library is already included in the Pipfile for this application, so we can try it out. Run `python debug.py` , and try out some Faker methods:

```
fake = Faker()  
fake.name()  
# => 'Samantha Taylor'  
fake.name()  
# => 'Connie Ferguson'  
fake.name()  
# => 'Christopher Ortega'
```

As you can see, every time we call the `name` method, we get a new random name. Faker has a lot of [built-in randomized data generators](https://faker.readthedocs.io/en/master/index.html%5D)  <https://faker.readthedocs.io/en/master/index.html%5D> that you can use:

```
fake.email()  
# => 'hubbardpatricia@example.com'  
fake.color()
```

```
# => '#7148af'  
fake.profile()
```

Let's use Faker to generate 50 random games (we will use the random library to generate prices). Replace the data after our data deletion in the `seed.py` file with the following code:

```
# seed.py  
  
# Add a console message so we can see output when the seed file runs  
print("Seeding games...")  
  
games = [  
    Game(  
        title=fake.name(),  
        genre=fake.word(),  
        platform=fake.word(),  
        price=random.randint(0, 60)  
    )  
    for i in range(50)]  
  
session.bulk_save_objects(games)  
session.commit()
```

► *What do we call the syntax that we used to create the `games` variable?*

Then, run `python seed.py` to reseed the database:

```
session.query(Game).count()  
# => 50  
session.query(Game).first()
```

```
# => Game(id=1, title="Lisa Barton", platform="hotel")  
session.query(Game)[-1]  
# => Game(id=50, title="Jesus Anderson", platform="meeting")
```

Great! Now we've got plenty of seed data to work with, and an easy way for ourselves or other developers to populate the database any time we need to do so.

Conclusion

In this lesson, we learned the importance of having a seed file along with our database migrations in order for ourselves and other developers to quickly set up the database with sample data. We also learned how to use the Faker library to quickly generate randomized seed data.

Resources

- [SQLAlchemy ORM Documentation](https://docs.sqlalchemy.org/en/14/orm/) ➞ (https://docs.sqlalchemy.org/en/14/orm/)
- [Faker Documentation](https://faker.readthedocs.io/en/master/index.html%5D) ➞ (https://faker.readthedocs.io/en/master/index.html%5D)
- [random — Generate pseudo-random numbers - Python](https://docs.python.org/3/library/random.html) ➞ (https://docs.python.org/3/library/random.html)